Faster `printf` Debugging (Part 2)

Nathan Jones

Contents

Introduction

Debugging via `printf` (or `Serial.print` or, as I suggested in a previous article, `snprintf`) can be an extremely useful tool for seeing, quickly, what your system is doing and to zero in on the parts of your system that are, or could soon be, causing errors. This tool is not without its downsides, however, and you may have found in your experimentation with `printf` that calling it more than a few times ***significantly* slows down your program**. This challenge limits the times you can use `printf/snprintf` IRL, despite how useful it can be. In this article (as we did in Part 1) we'll do our best to **make `printf/snprintf` blisteringly fast** so that you can use it to your heart's content!

At the end of the last article, we performed a handful of optimizations that reduced the time it took a Nucleo-F042K6 to send out a 22-character message (with one integer argument) from **7,030 µs** to **140 µs** (a **50.2x speed-up**!). These optimizations were:

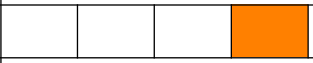
- Increase the processor's clock speed from 8 MHz to 48 MHz
- Increase the UART baud rate from 38400 to 3M
- Replace the STM32 HAL with the LL drivers

Additionally, we discussed a few ways to circumvent the problem entirely, which were:

- Make fewer calls to `printf/snprintf`
- Simplify your calls to `printf/snprintf`
- Shorten your messages
- Simulate your system

In this article, we'll look at a few more advanced optimization techniques that can further reduce our total message time from 140 µs to **88 µs**!

Use a non-blocking form of message transmission

Category	"Circumvent the Problem"
Difficulty	
Time savings	20 µs (+a few µs freed up)

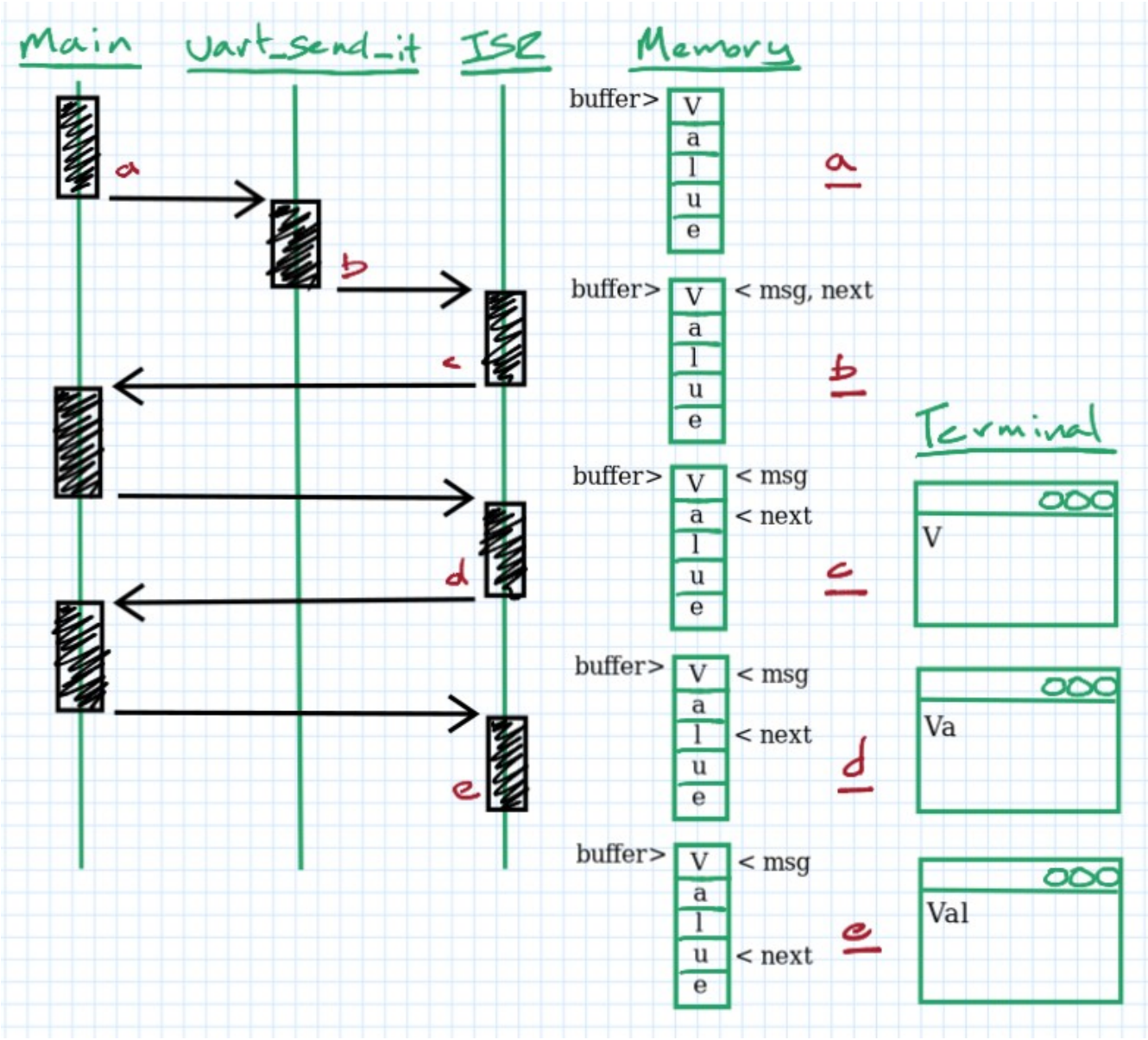
Part of the reason that `printf` seems like it's hogging our system is because it's set up that way! It's a **blocking** function, meaning that even while the processor is waiting for your message to be sent out the UART port, `printf` *won't* let it do anything else.

We can let our processor do other useful work by using `snprintf` to format our code and then use either the microcontroller's interrupt or its DMA system to do the actual sending of our code. Here's how that might look using the STM32 LL library for the sample code I showed above:

```
// main.c
//
#define MSG_LEN 128
char buffer[MSG_LEN+1] = {0};
snprintf(buffer, MSG_LEN, “Value of counter: %d\n”, counter++);
uart_send_it(buffer);
```

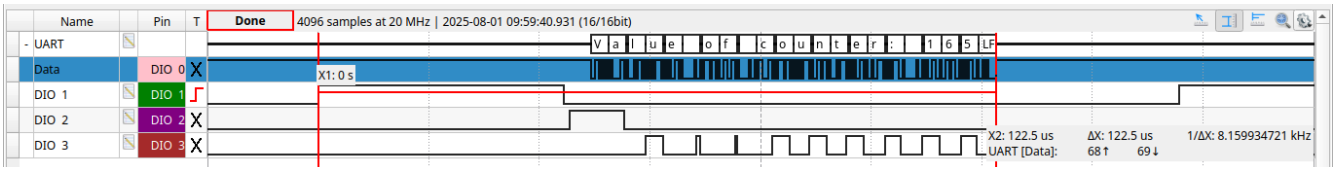
The `uart_send_it()` function is one I wrote; it stores the pointer to the message into a local variable and turns on the UART transmit ISR. Every time the ISR runs, it will copy a new byte into the UART transmit buffer until it reaches the NUL terminator.

```
// uart.c
//
static char *msg = NULL, *next = NULL;

bool uart_send_it(char * new_msg)
{
  bool success = false;
  if(new_msg && NULL == msg)
  {
    msg = next = new_msg;
    LL_USART_EnableIT_TXE(USART2);
    success = true
  }
  return success;
}

void USART2_IRQHandler(void)
{
  if(next && *next) LL_USART_TransmitData8(USART2, *next++);
  else
  {
    msg = next = NULL;
    LL_USART_DisableIT_TXE(USART2);
  }
}
```



a: A message is stored in `buffer` and `uart_send_it()` is called.
b: `uart_send_it()` sets `msg` and `next` pointers to the start of the message and turns on the UART ISR.
c: The UART ISR is immediately called. ‘V’ is printed to the terminal and `next` is advanced by one.
d: The ISR runs again; ‘a’ is printed and `next` advances by one.
e: The ISR runs again; ‘l’ is printed and `next` advances by one.

The end result is that `uart_send_it()` is very short and completes quickly, letting the processor get on with other work while the first byte gets shifted out. As each byte is sent out, the processer gets momentarily interrupted to load the next byte of the message into the UART Tx register and then it gets to return to whatever it was previously doing.



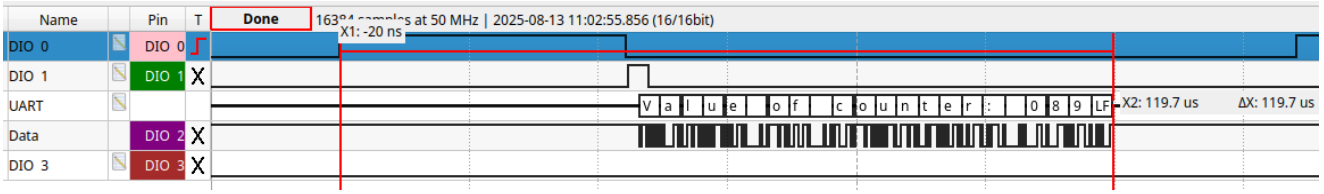
The logic analyzer image above shows this system at work. DIO1 is toggling around the call to `snprintf`, DIO2 is toggling around the call to `uart_send_it`, and DIO3 is toggling while the UART is sending out our message (representing the other work our processor is trying to do).

It only took **123 us** to send out our message (which, interestingly, is faster than the 140 us it took using `printf` in our last example, even though the messages are formatted the same, possibly because `snprintf` runs *slightly* faster than `printf`). As importantly, we can see DIO3 toggling *while* our message is being sent, which represents other work our processor can be doing during message transmission. The biggest problem seems to be that there aren't a *lot* of toggles above; in other tests that I ran, the processor was only able to increment an integer 23 times while the message above was being sent out, which is not a lot of other work that it can do. But its not nothing! And we can improve this value slightly further by **compiling at a higher optimization level**.

Compile at -O3

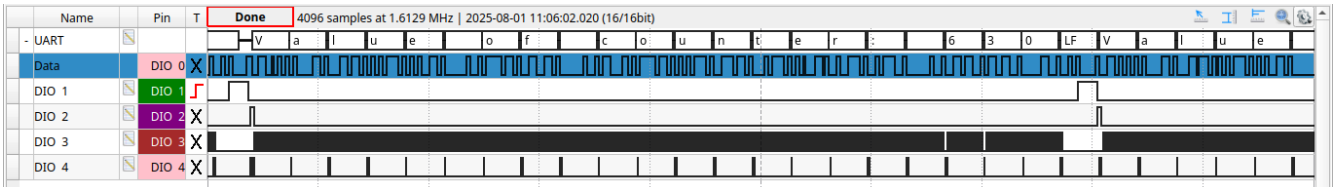
Category	"Circumvent the Problem"
Difficulty	
Time savings	3 µs (+57 µs freed up)

Now that we're actually compiling our own code (the newlib-nano versions of `printf` and `snprintf` that we were using before came pre-compiled), increasing the optimization level of our compiler from -O0 (none; the default) to -O3 (highest optimization) can have positive effects on our code! Compiling our example above at -O3 reduces the total message time (just slightly) to **120 µs**. More importantly, look at how much more often DIO3 is getting to toggle!



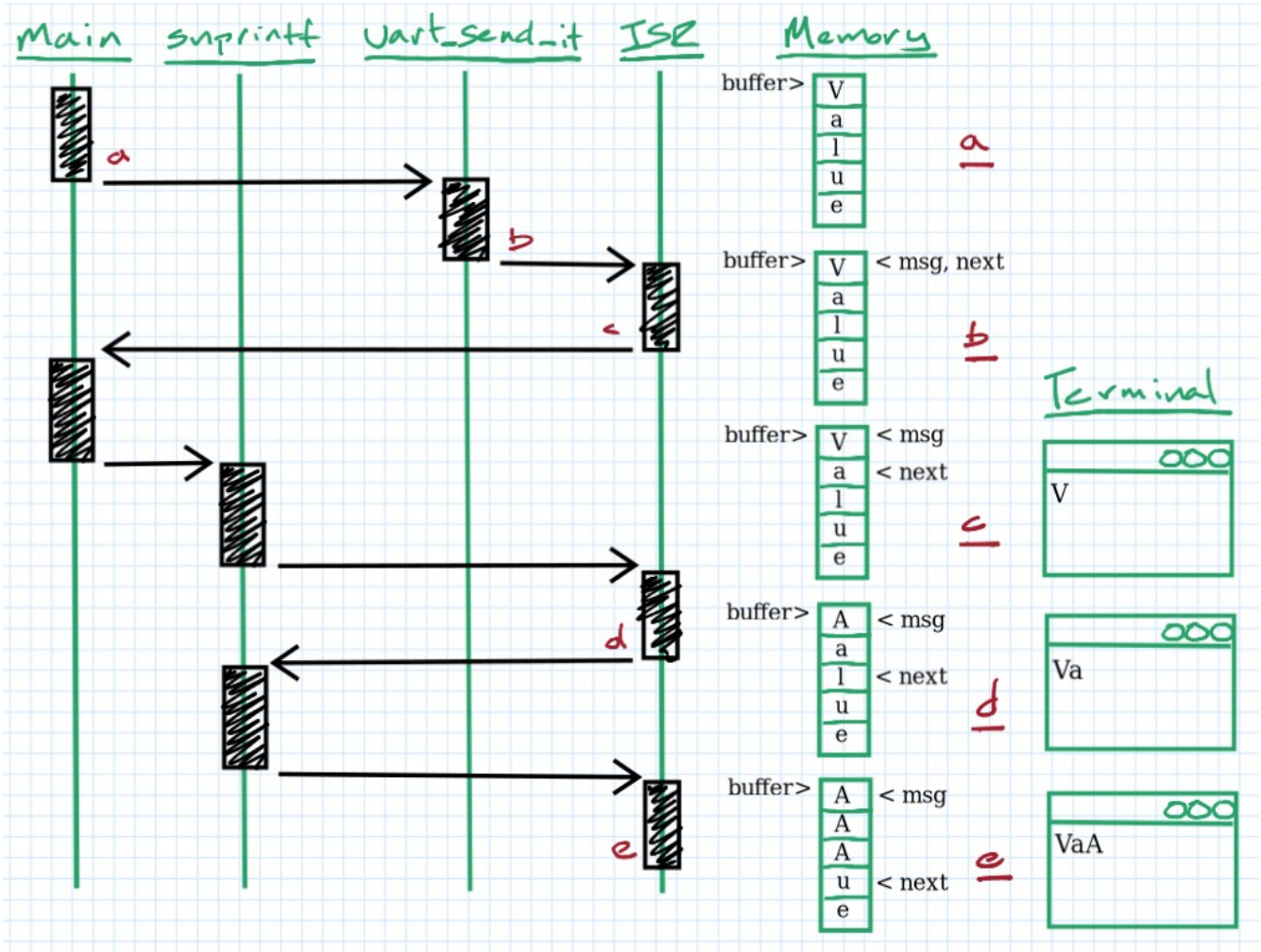
In my tests, I was able to increment an integer 89 times while our message was being sent (up from 23 times), which means our processor can do nearly **4 times as much work** while this message was being sent as when we were only compiling at -O0. In total, **the processor was only busy during this time for a total of about 63 µs**.

Okay, back to our non-blocking message transmissions! Although we saw above that not a *lot* of work was being done while our message was being sent, it becomes much more important if our system *can't* actually send UART data at 3 Mbaud. If we were limited to, say, the much more standard value of 115200 baud, we could see DIO3 toggling *a lot* during our message transmission.



This represents a *ton* of useful work that our processor is able to do while our UART peripheral is slowly shifting out our message at the sedate pace of 115200 bits per second.

This is, however, not a *complete* solution since we have a bit of a **synchronization problem**. Assuming the code above is executed more than once, when `snprintf` runs again we have no way of knowing if the UART is actually finished sending the *first* message! We'll accidentally clobber that first message if we let `snprintf` execute before the previous message is finished being sent.



a-c: As before.
d: `snprintf` begins copying the next message (“AAAAA…”) into `buffer`, though it hasn’t caught up to where `next` is, yet. The UART ISR runs and prints ‘a’ to the terminal, advancing `next` by one.
e: `snprintf` continues to fill in `buffer` with the new message and has, at this time, surpassed `next`. The UART ISR runs, printing ‘A’ to the terminal now, since the old message has now been overwritten.

One simple fix is to wait until the UART is done sending our message before we make any calls to `snprintf`. This will require a small helper function, `uart_using()`, that will tell us if any message currently being transmitted matches the buffer we're trying to use to send out our *next* message.

```
// main.c
//
while(uart_using(buffer));
snprintf(buffer, MSG_LEN, “Value of counter: %d\n”, counter++);
uart_send_it(buffer);

// uart.c
//
bool uart_using(char * thisMsg)
{
  return (thisMsg == msg);
}
```

The while loop above will block for as long as the UART peripheral is reading from `buffer`, preventing any overwrite errors.

For the curious, a few other options to fix our synchronization problem are listed at the end of this section.

Of course, there's *also* always a chance that the UART peripheral is busy sending another message and *doesn't* accept ours when we call `uart_send_it`, returning a “success” value of `false` instead of `true`. If you're okay dropping the occasional message, then you don't need to change your code. **If you're not okay with dropping any messages, though, you'll need to block until the UART becomes free**

```
while(uart_using(buffer));
snprintf(buffer, MSG_LEN, "Value of counter: %03d\n", (int)counter++);
while(!uart_send_it(buffer));
```

or add some form of buffer to the HAL code so that it can store pointers to multiple messages at a time. This would allow the HAL code to accept more than one message pointer at a time, reducing (or, hopefully, eliminating!) any client code from actually waiting in that `while` loop above for the UART to free up. This would also help if your messages came in bursts, since the UART could receive messages in a "bursty" fashion (up to the limit of its internal buffer) while transmitting them at a slower pace.

This just scratches the surface of designing a system that communicates by sending pointers to messages between its tasks, though. For a more thorough discussion, see "Message Passing with Pointers".

Use dynamic memory

```
const size_t MSG_LEN = 64;
char * msg = malloc(sizeof(char)*(MSG_LEN+1));
if(msg)
{
```

```
  snprintf(msg, MSG_LEN, ...);
  uart_send_it(...);
}
else
{
  //Fatal error? Out of heap space
}
```

To avoid a memory leak, the UART ISR should then call `free()` on the message pointer once it was finished transmitting.

Double-buffer the messages

```
const size_t MSG_LEN = 64;
char msg[2][MSG_LEN+1] = {0};
size_t using = 0;

snprintf(msg[using], MSG_LEN, …);
while(!uart_send_it(msg[using]));
using = 1 - using;
```

This takes advantage of the fact that the HAL code can only accept a single message pointer at a time. Once the call to `uart_send_it()` returns with `true`, it means it *must* be finished sending the other message in our array (if, indeed, it was even sending that one at all).

Give a "free" callback to the UART code

```
static boolean freed = true;
while(!freed);
freed = false;
snprintf(…);
uart_send_it_with_cbk(buffer, myFree);

// Elsewhere in main.c
//
void myFree(){ freed = true; }
```

The UART ISR would then call the callback function (`myFree()`, in this case) once the message was done being sent. The task would block on the `freed` variable if it was called again until the callback was complete.

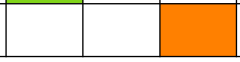
The added complexity gains us slightly better performance; our tasks now only need to wait to call `snprintf` if the UART is sending *their* message, not *any* message, like before.

Request message pointer from UART HAL

In this solution, the UART HAL allocates storage for all of the messages to be sent; none of the tasks do. Tasks "request" a pointer to a spot in memory to place a debug message, which the UART HAL gives out as long as it has space to give.

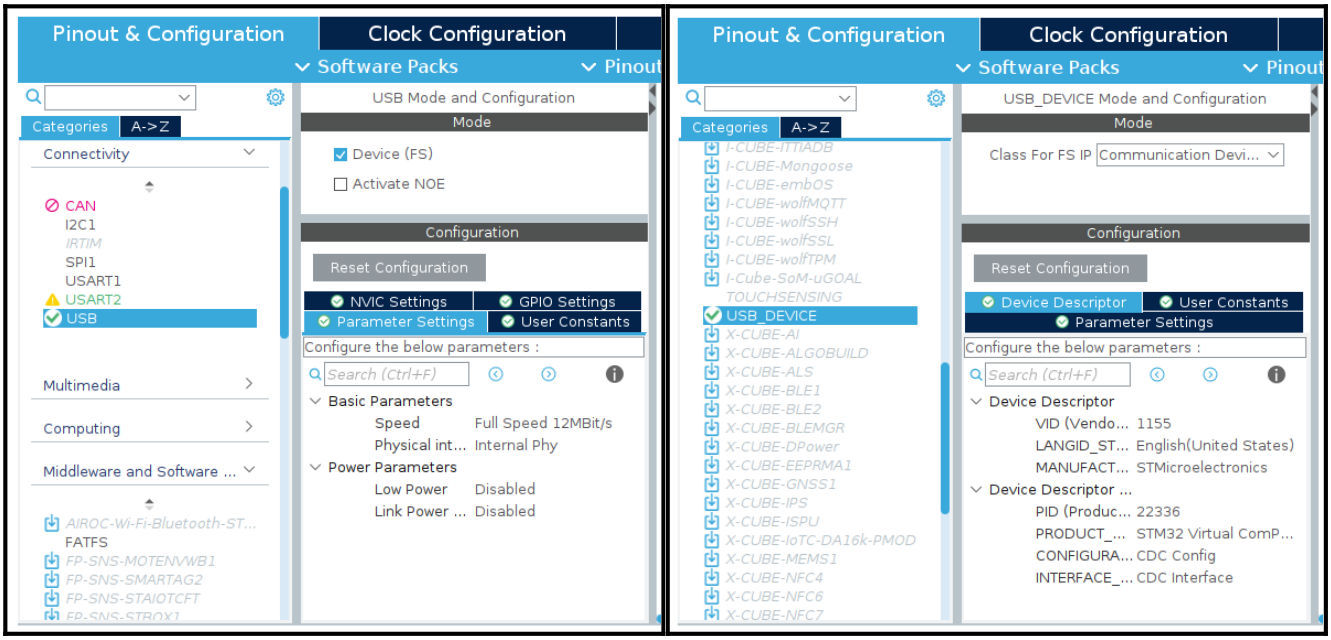
```
// task.c
//
char * msg = getPtr();
if(msg)
{
  snprintf(msg, MSG_LEN, ...);
  uart_send_it(...);
}

// uart.c
//
const size_t MSG_LEN = 64;
char msg[MSG_LEN+1] = {0};
bool in_use = false;

char * getPtr(void)
{
  if(!in_use)
  {
    in_use = true;
    return &msg;
  }
  else return NULL;
}

void ISR(void)
{
  // ...
  if( /* Done sending msg */ )
  {
    in_use = false;
    memset(msg, 0, MSG_LEN);
  }
}
```

Use USB

Category	“Transmit Faster”
Difficulty (ST)	
Difficulty (others)	
Time savings	Up to 38 µs faster (but maybe 87 µs slower)

UART is, of course, not the only form of serial communication at our disposal. The STM32F042K6 on my Nucleo board happens to have a USB peripheral, which supports USB FS (Full Speed), with transfer rates as high as **12 Mbit/sec** or 4x faster than our fastest UART transfer rate of 3 Mbaud!

Once configured as a CDC (Communication Device Class) USB device, sending and receiving messages is about as straightforward as it is using UART. The catch, though, and the reason this technique is listed as higher difficulty than UART, is that you'll need a USB library to manage things like device enumeration, connection management, and the low-level transfer of ASCII characters. ST provides one such library, USB_DEVICE, which is easily configured inside STM32CubeIDE once the USB peripheral has been enabled.



If you're not using an ST microcontroller, though, you'll need to find your own USB library, which may make this technique more difficult for you than others.

The ST USB library provides an easy function for sending byte data out the USB port: `CDC_Transmit_FS(uint8_t* Buf, uint16_t Len)`. So our USB code would look like this:

```
// main.c
//
#define MSG_LEN 128
char buffer[MSG_LEN+1] = {0};
snprintf(buffer, MSG_LEN, "Value of counter: %d\n", counter++);
CDC_Transmit_FS((uint8_t*)buffer, strlen(buffer));
```

NOTE: The USB stack implemented by this library is large and would normally exceed the RAM space I have available on the Nucleo-F042K6. The only way I could make this example fit was to reduce the size of `APP_RX_DATA_SIZE` (defined on line 52 of `USB_DEVICE/App/usbd_cdc_if.h`) to 0 (since we're not receiving any data from the host computer).

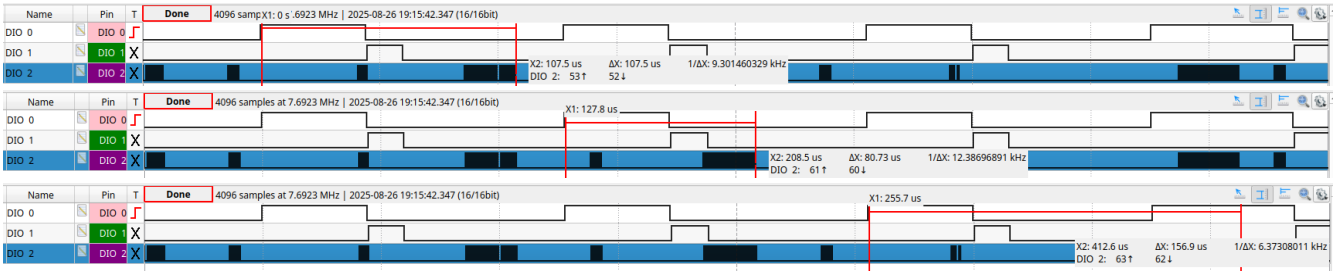
Of note is that `CDC_Transmit_FS()` operates **asynchronously**, using the USB peripheral's interrupt system to send out your message. This means that, as discussed above, we have a

synchronization problem since it's *possible* for our system to overwrite `buffer` *while* it's being transmitted if this section of code were to be called a second time (or more) before the USB peripheral were to finish transmitting our message. In my testing, that threshold occurred once the message was longer than about 64 characters. At that point, the message took long enough to send out over USB that a subsequent call to `sprintf` (or, really, `memset`) overwrote the end of the message before it could be processed by the USB peripheral. AFAIK, the USB_DEVICE library doesn't offer us a convenient way to check if it's busy (other than as a return value associated with trying to send *another* message), so the best solution to this problem seems to use our own double-buffer (shown below), as mentioned here, filling one side of the buffer while the other side is being transmitted. Additionally, we'll enclose the call to `CDC_Transmit_FS()` in a `while` loop to ensure that no message gets dropped if the USB peripheral were to be busy at the moment we tried to send a message.

```
// main.c
//
#define MSG_LEN 128
char msg[2][MSG_LEN+1] = {0};
size_t using = 0;
snprintf(msg[using], MSG_LEN, …);
while(USBD_OK != CDC_Transmit_FS((uint8_t*)msg[using], strlen(msg[using])));
using = 1 - using;
```

I used this USB cable, connected to pins D2 and D10 on the Nucleo-F042K6, to receive the USB messages on my laptop. (The physical USB port on the Nucleo board is connected to the on-board debug adapter, which allows for programming, single-step debugging, and UART communication through a virtual COM port to which the USART2 peripheral is connected. The USB peripheral is *not* connected here and, instead, is only brought out to the two pins listed above.)

In my testing, it only took about **22 µs** to send out our message, a speed-up of about **3.4x** as compared to UART at 3 Mbaud (this is slightly less than the 4x speed-up we expected because USB includes various control messages, bit stuffing, and a CRC in each message). In total, then, you might think that it will only take **82 µs** to send out our sample debug message over USB (including the time taken to format the string and call `CDC_Transmit_FS()`), which would make this technique 38 µs *faster* than high-speed UART. In reality, though, the full message time is much longer, possibly taking up to **207 µs** for each message (which would actually make USB 87 µs *slower* than UART).



This variability comes from the fact that USB is a *polled* or *host-driven* form of serial communication. In other words, our device can't *push* data to the host computer, it can only be *pulled* upon a request

from the host computer. This only happens once per millisecond for USB FS and once per 125 µs for USB HS. Indeed, our messages above were only being sent out *exactly* every 125 µs (despite our microcontroller only being configured for USB FS, possibly owing to the USB hub to which I've connected my USB cable being a High Speed device and so querying USB endpoints at the HS rate of once per 125 µs). What this means is that *after* our message has been formatted and `CDC_Transmit_FS()` called, we need to wait a random interval between 0 and 125 µs for the next query from the host computer. We can amortize this cost, though (asymptotically approaching that ideal message time of 82 µs), if we buffer our own messages, calling `CDC_Transmit_FS()` only when we have a large number of messages to send.

Buffer it

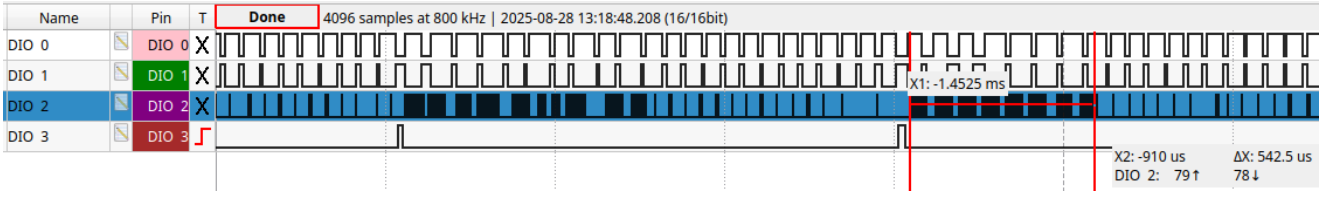
Category	"Transmit Faster"
Difficulty	
Time savings	32 µs

If our USB messages have to align to 125 µs microframes and if we have to pay a cost in control messages and CRCs for each USB packet sent, we can limit the effect of these accumulated delays on our transmissions by storing up messages and sending them out only once we've gathered a large number of them. Instead of calling `CDC_Transmit_FS()` for each message, we'll call a function we create called `post()` that copies a message to an internal buffer.

```
// main.c
//
#define MSG_LEN 128
char buffer[MSG_LEN+1] = {0};
snprintf(buffer, MSG_LEN, "Value of counter: %d\n", counter++);
post((uint8_t*)buffer, strlen(buffer));
```

`post()` will call `CDC_Transmit_FS()` once its buffer is filled. (Implementation of `post()` is an exercise left to the reader! It's recommended, though, to use a double-buffer in your implementation, for the same reasons mentioned previously.)

Despite some variability in the execution time of `snprintf`, in my tests the total transmission time for each of our debug messages did, in fact, average out to **88 µs**[1], making this technique **32 µs faster** than high speed UART!



1: 543 µs to send out 23 messages is 24 µs per message. Add to that the average time for `snprintf` (50 µs, oddly, for this test) and the average time to call `post()` (14 µs) and you get a total message

time of 88 µs.

Conclusion

In this article we employed **asynchronous message transmissions** and a **buffered USB channel** to cut our debug message time down from 140 µs to **88 µs**, which is nearly **80x faster** than our baseline!

Example	Difficulty	Function	Clock (MHz)	Baud rate	HAL / LL	Opt. Level	Formatting (µs)	Transmitting (µs)	Total time[1] (µs)
Baseline	N/A	`printf`	8	38400	HAL	-O0	365	6,668	**7,030**
Faster clock		`printf`	48	38400	HAL	-O0	68	5,895	**5,963**
Faster baud		`printf`	48	3000000	HAL	-O0	68	235	**303**
Ditch the HAL		`printf`	48	3000000	LL	-O0	65	75	**140**
Async		`snprintf`	48	3000000	LL	-O0	45+3[2]	75	**123**[3]
-O3		`snprintf`	48	3000000	LL	-O3	45	75	**120**[3]
USB		`snprintf`	48	12000000	N/A	-O3	45+15[2]	22-147	**82-207**[3]
Buffer It		`snprintf`	48	N/A	N/A	-O3	50+14[2]	24	**88**[3]

1: Time to send a 22 character message with one integer
2: First time listed is the execution time of `snprintf`; the second is the time it took to put our debug message in whatever buffer was being used to asynchronously send out our data.
3: Processor only busy during this time for a total of about 63 µs.

These techniques are significantly more challenging than those discussed in the last article, though. Remember that faster message times just for the sake of faster message times is a waste of developer effort; I encourage you to always evaluate when “fast” is “fast enough” to decide if such techniques are worth your time.

That being said, in the next article we’ll pull out all the stops to make `printf` *even faster*! Stay tuned for even more speed.

If you’ve made it this far, thanks for reading and happy hacking!